

# Agent Workflow Protocol: A Declarative, Runtime-Agnostic Standard for Portable Multi-Agent AI Workflows

**Authors:** AWP Working Group

**Date:** March 2026

**Version:** 1.0.0

---

## Abstract

---

The rapid adoption of large language model (LLM)-based agents has created a fragmented ecosystem in which multi-agent workflows are tightly coupled to proprietary frameworks and runtimes. Existing standards — MCP for tool access, A2A for agent communication, OpenAPI for HTTP APIs — each address isolated concerns, leaving teams to invent ad-hoc glue for orchestration, state management, and observability. We present the **Agent Workflow Protocol (AWP)**, an open, declarative standard that describes complete multi-agent workflows in a single, runtime-agnostic YAML manifest. AWP introduces a 7-layer architecture covering manifest metadata, agent identity, capabilities, communication, memory and state, orchestration, and observability. It defines five autonomy levels (A0–A4) ranging from prescribed static pipelines to fully self-organizing agent teams with recursive delegation and hard budget enforcement. We provide a formal specification with 24 deterministic validation rules, a Python reference implementation, 12 example workflows, and a conformance test suite. Preliminary evaluation demonstrates that AWP manifests are portable across runtimes, enable pre-execution validation that eliminates a class of runtime errors, and support workflows spanning the full autonomy spectrum without format changes.

**Keywords:** multi-agent systems, workflow orchestration, protocol specification, large language models, declarative programming, interoperability

---

## 1. Introduction

---

### 1.1 Background

The emergence of LLM-based autonomous agents has shifted the AI engineering paradigm from single-model inference to multi-agent collaboration. Systems such as AutoGen (Wu et al., 2023), CrewAI (Moura, 2024), and LangGraph (LangChain, 2024) enable developers to compose multiple agents into workflows where each agent contributes specialized capabilities.

However, this rapid innovation has produced significant fragmentation. Each framework introduces its own configuration format, orchestration model, state management strategy, and tool integration approach. A workflow designed for CrewAI cannot run on LangGraph without a complete rewrite. This vendor lock-in impedes portability, benchmarking, and knowledge sharing across the community.

## 1.2 The Interoperability Gap

Several open standards have emerged to address specific aspects of the agent ecosystem:

- **Model Context Protocol (MCP)** (Anthropic, 2024) standardizes how LLMs access external tools but defines no agent identity, orchestration model, or state management.
- **Agent-to-Agent Protocol (A2A)** (Google, 2025) specifies inter-agent communication but lacks orchestration primitives, memory protocols, and tool definitions.
- **OpenAPI** (OpenAPI Initiative, 2024) describes HTTP APIs but has no concept of agents, execution graphs, or shared state.

Table 1 summarizes the coverage of existing standards across the six core concerns of multi-agent workflows.

| Concern          | MCP  | A2A     | OpenAPI | LangGraph | CrewAI  | AWP  |
|------------------|------|---------|---------|-----------|---------|------|
| Agent Identity   | -    | Partial | -       | -         | Partial | Full |
| Tool Integration | Full | -       | Full    | Partial   | Partial | Full |
| Communication    | -    | Full    | -       | -         | Partial | Full |
| State Management | -    | -       | -       | Full      | -       | Full |
| Orchestration    | -    | -       | -       | Full      | Partial | Full |
| Observability    | -    | -       | -       | -         | -       | Full |

No single standard provides complete coverage. AWP is designed to fill this gap.

## 1.3 Contributions

This paper makes the following contributions:

1. **A 7-layer declarative architecture** for describing multi-agent workflows completely and runtime-agnostically (Section 3).
2. **A formal autonomy taxonomy** (A0–A4) that classifies workflows along a spectrum from prescribed to self-organizing (Section 4).
3. **Two complementary orchestration engines** — DAG and delegation loop — supporting the full autonomy spectrum (Section 5).

4. **A validation framework** with 24 deterministic pre-execution rules that eliminate a class of runtime errors (Section 6).
  5. **A reference implementation** in Python with CLI tooling, parser, validator, and dual-engine runtime (Section 7).
  6. **A conformance test suite** enabling third-party runtimes to verify AWP compliance (Section 8).
- 

## 2. Design Principles

---

AWP is governed by eight design principles that constrain all specification decisions.

**P1: Declarative Over Imperative.** AWP manifests describe *what* a workflow is, not *how* to execute it. Agent graphs, tool bindings, and communication channels are declared as data. Runtime strategy is an implementation concern.

**P2: Runtime-Agnostic.** No assumption is made about the implementation language or execution environment. The same `workflow.awp.yaml` must produce equivalent behavior on any conforming runtime.

**P3: Composable.** Workflows may include sub-workflows. An agent in one workflow can be an entire workflow in another. This enables hierarchical composition from well-tested components.

**P4: Versioned.** Every manifest carries a SemVer version. Runtimes check compatibility before execution. Breaking changes increment the major version.

**P5: Self-Contained.** A packaged AWP workflow (`.awp.zip`) contains all artifacts needed for execution: manifest, agent definitions, prompt files, output schemas, skills, and project-local tools. No external dependencies beyond the runtime and LLM provider credentials.

**P6: Progressive.** The minimum valid workflow requires fewer than 10 lines of YAML. Advanced features are optional and additive. Complexity is adopted incrementally.

**P7: Explicit Over Implicit.** Every dependency, shared field, and tool permission is declared in the manifest. There are no hidden defaults that alter behavior.

**P8: Secure by Default.** Agents operate under the principle of least privilege. Tools must be explicitly allowed per agent. Shell execution is disabled unless declared. File access is scoped to the workflow directory.

---

## 3. The 7-Layer Architecture

---

AWP structures a multi-agent workflow as a stack of seven layers. Each layer depends on layers below but not above. Layers 0 and 1 are mandatory; all others are optional.

### 3.1 Layer 0: Manifest

The manifest layer provides workflow-level metadata: name, version, description, author, tags, and the AWP protocol version. It serves as the entry point for parsing and version compatibility checking.

```
awp: "1.0.0"
workflow:
  name: research-pipeline
  version: "1.0.0"
  description: "Multi-agent research pipeline"
```

### 3.2 Layer 1: Agent Identity

Each agent is defined in a separate `agent.awp.yaml` file specifying its name, role description, LLM configuration (model, temperature, token limits), system prompt, and output schema.

### 3.3 Layer 2: Capabilities

The capabilities layer declares what tools an agent may use (via MCP tool references), what skills it possesses, what data sources it can access, and what sandbox constraints apply. Tool permissions are per-agent, enforcing the principle of least privilege.

### 3.4 Layer 3: Communication

The communication layer defines typed message channels and a message bus for inter-agent messaging. Messages are wrapped in typed envelopes with sender, recipient, timestamp, and payload schema.

### 3.5 Layer 4: Memory and State

The memory layer specifies the state model (shared dictionary, scoped, or hierarchical), sharing strategy (full, selective, or isolated), persistence tiers (ephemeral, session, long-term), and output contracts that define what each agent must produce.

### 3.6 Layer 5: Orchestration

The orchestration layer defines the execution model. AWP supports two engines:

- **DAG Engine:** A static directed acyclic graph with topological execution order, conditional branching ( `when` expressions), and fan-out parallelism.
- **Delegation Loop Engine:** A dynamic manager-worker loop where the manager agent spawns ephemeral workers at runtime with generated instructions, tool allowlists, and output contracts.

### 3.7 Layer 6: Observability

The observability layer specifies tracing configuration, metric collection points, health check endpoints, and audit log requirements. At autonomy level A4, observability is a compliance requirement, not optional.

---

## 4. Autonomy Taxonomy

---

AWP introduces a five-level taxonomy for classifying the autonomy of multi-agent workflows. Autonomy levels are orthogonal to feature adoption — a simple A0 workflow may use all seven layers, while a complex A4 workflow must meet specific compliance requirements.

### 4.1 Level Definitions

**A0 — Prescribed.** All agents, tools, and execution paths are statically defined. The DAG is fixed. No runtime decisions.

**A1 — Adaptive.** Static agent set with conditional execution. Agents may be skipped based on runtime state (when: `state.score > 0.7`). Loops and fan-out are permitted.

**A2 — Delegating.** A manager agent dynamically spawns worker agents at runtime. Workers receive generated instructions and tool configurations via delegation envelopes. Hard budget constraints are required.

**A3 — Self-Tooling.** Agents may create new tools or skills at runtime using a dynamic tool factory. Created tools are validated against security policies before activation.

**A4 — Self-Organizing.** Full recursive delegation. Manager agents may delegate to sub-managers who further delegate. Budget is distributed hierarchically. Observability is mandatory. Stagnation detection with automatic termination is required.

### 4.2 Compliance Requirements

Each level imposes cumulative requirements. Table 2 summarizes key constraints.

| Requirement           | A0       | A1       | A2       | A3       | A4       |
|-----------------------|----------|----------|----------|----------|----------|
| Static agent graph    | Required | Required | -        | -        | -        |
| Budget enforcement    | -        | -        | Required | Required | Required |
| Delegation envelopes  | -        | -        | Required | Required | Required |
| Dynamic tool creation | -        | -        | -        | Allowed  | Allowed  |

| Requirement          | A0       | A1       | A2          | A3       | A4       |
|----------------------|----------|----------|-------------|----------|----------|
| Recursive delegation | -        | -        | -           | -        | Allowed  |
| Observability        | Optional | Optional | Recommended | Required | Required |
| Stagnation detection | -        | -        | Optional    | Optional | Required |

## 5. Orchestration Engines

### 5.1 DAG Engine

The DAG engine executes agents in topological order derived from the declared dependency graph. It supports:

- **Sequential and parallel execution** based on dependency structure
- **State sharing** via `share_output` declarations that inject upstream outputs into downstream agent contexts
- **Conditional execution** via `when` expressions evaluated against shared state using a safe expression evaluator (AST-based, no `eval()`)
- **Error handling** with configurable retry policies and circuit breakers

Formal property: Given a valid DAG  $G = (V, E)$  where  $V$  is the set of agents and  $E$  the set of dependencies, the DAG engine guarantees that for every edge  $(u, v)$  in  $E$ , agent  $u$  completes before agent  $v$  begins execution. Cycle detection (validation rule R12) ensures  $G$  is acyclic.

### 5.2 Delegation Loop Engine

The delegation loop engine implements a manager-worker pattern for dynamic orchestration:

1. The manager agent receives the task and current state.
2. The manager produces a **delegation envelope** specifying: worker instructions, allowed tools, output contract, skill assignments, and code execution permissions.
3. Workers execute in parallel within enforced sandbox constraints.
4. Results are validated (deterministically and optionally via LLM).
5. The manager reviews results and either terminates or initiates another loop iteration.

Budget enforcement is mandatory. The engine tracks: - `max_loops`: Maximum iteration count - `max_total_workers`: Total worker instances across all loops - `max_total_tokens`: Aggregate token consumption - `max_wall_time`: Wall-clock timeout in seconds - `max_tool_calls`: Total tool invocations - `max_depth`: Maximum recursive delegation depth (A4)

Stagnation detection monitors confidence scores over a rolling window. If the confidence delta falls below `min_confidence_delta` for `window` consecutive loops, the engine issues a warning and optionally terminates.

### 5.3 Engine Composition

AWP permits DAG and delegation loop composition within a single workflow. A DAG node may reference a delegation loop sub-workflow, enabling patterns such as: static planning phase (DAG) → dynamic research phase (delegation loop) → static reporting phase (DAG).

---

## 6. Validation Framework

---

AWP defines 24 deterministic validation rules (R1–R24) that are checked before execution begins. Rules are grouped by concern:

**Structural Rules (R1–R5):** Manifest schema conformance, protocol version compatibility, required field presence, valid SemVer format, and unique workflow naming.

**Agent Rules (R6–R10):** Agent file existence, prompt file references, LLM configuration validity, output schema well-formedness, and agent-graph consistency.

**Graph Rules (R11–R15):** Dependency target existence, cycle detection, root node presence, connected graph verification, and topological sort feasibility.

**Contract Rules (R16–R20):** Output field declaration, `share_output` field coverage, downstream dependency satisfaction, type compatibility, and schema validation.

**Security Rules (R21–R24):** Tool permission scoping, sandbox configuration validity, budget constraint presence (A2+), and observability requirement (A4).

The validation framework enables a key property: **if a workflow passes all 24 rules, it is guaranteed to be structurally executable by any conforming runtime.** This eliminates a class of runtime errors related to missing files, broken dependencies, cyclic graphs, and permission violations.

---

## 7. Reference Implementation

---

### 7.1 Architecture

The Python reference implementation comprises five subsystems:

1. **Parser** (`awp.parser`): YAML-to-Pydantic model conversion for manifests and agent definitions. Uses strict type checking via Pydantic v2 discriminated unions.

2. **Validator** (`awp.validator`): Implements all 24 validation rules plus schema validation against JSON Schema definitions. Returns structured validation results with error locations and fix suggestions.
3. **Runtime** (`awp.runtime`): Dual-engine executor supporting both DAG and delegation loop orchestration. Includes LLM client abstraction (OpenRouter, OpenAI, Ollama), message bus, sandboxed code executor, and dynamic tool factory.
4. **Packager** (`awp.packager`): Creates and extracts `.awp.zip` archives containing all workflow artifacts. Validates package integrity on extraction.
5. **CLI** (`awp.cli`): Command-line interface exposing `validate`, `run`, `pack`, `unpack`, `visualize`, `compliance`, and `identity-card` subcommands.

## 7.2 CLI Usage

```
# Validate workflow against all 24 rules
awp validate path/to/workflow/

# Check autonomy level compliance
awp compliance path/to/workflow/ --level A2

# Execute workflow with task
awp run path/to/workflow/ --task "Analyze market trends"

# Package for distribution
awp pack path/to/workflow/ -o workflow.awp.zip

# Visualize execution graph
awp visualize path/to/workflow/ --format mermaid
```

## 8. Evaluation

### 8.1 Test Suite

The reference implementation includes 15 test modules with coverage across all subsystems:

| Test Module                 | Scope                                 | Tests |
|-----------------------------|---------------------------------------|-------|
| <code>test_parser</code>    | YAML parsing for all 7 layers         | 22    |
| <code>test_validator</code> | 24 validation rules (valid + invalid) | 34    |
| <code>test_models</code>    | Pydantic model behavior               | 18    |
| <code>test_runtime</code>   | DAG runner core logic                 | 12    |



| Test Module              | Scope                                           | Tests |
|--------------------------|-------------------------------------------------|-------|
| test_e2e                 | Full pipeline: parse → validate → pack          | 8     |
| test_delegation_loop_e2e | Manager-worker loop execution                   | 6     |
| test_examples_e2e        | All 12 example workflows                        | 12    |
| test_code_executor       | Sandboxed Python execution                      | 8     |
| test_expressions         | Safe <code>when</code> expression evaluation    | 10    |
| test_message_bus         | Inter-agent messaging                           | 6     |
| test_observability       | Tracing and metrics                             | 5     |
| test_security_runtime    | Circuit breaker, rate limiting                  | 7     |
| test_secrets             | Secret injection from <code>secrets.yaml</code> | 4     |
| test_enterprise_e2e      | Complex enterprise workflows                    | 3     |
| test_conformance         | Standardized conformance suite                  | 15    |

## 8.2 Conformance Test Suite

The conformance test suite (`conformance/suite.json`) provides a standardized set of valid and invalid workflow fixtures. Any AWP-compliant runtime must:

1. **Accept** all valid fixtures without errors.
2. **Reject** all invalid fixtures with the specified error codes.

Invalid fixtures cover: cyclic dependency graphs, missing required fields, reserved namespace violations, orphaned agents, invalid SemVer versions, and budget constraint violations.

## 8.3 Example Workflows

Twelve example workflows demonstrate the full autonomy spectrum:

| Example                 | Autonomy | Agents | Key Feature                  |
|-------------------------|----------|--------|------------------------------|
| 01-hello-world          | A0       | 1      | Minimal valid workflow       |
| 02-research-pipeline    | A1       | 3      | DAG with state sharing       |
| 03-chat-team            | A1       | 3      | Message bus communication    |
| 04-memory-workflow      | A1       | 3      | Long-term memory persistence |
| 05-observable-analytics | A1       | 3      | Tracing and metrics          |
| 06-enterprise           | A1       | 4      | Security, skills, code mode  |

| Example                 | Autonomy | Agents  | Key Feature                       |
|-------------------------|----------|---------|-----------------------------------|
| 07-dynamic-tools        | A3       | 3       | Runtime tool creation             |
| 08-delegation-loop      | A2       | Dynamic | Manager-worker delegation         |
| 09-recursive-delegation | A4       | Dynamic | Recursive sub-delegation          |
| 10-skill-generation     | A3       | Dynamic | Skill creation in delegation loop |
| 11-tool-creation-loop   | A3       | Dynamic | Scoring tool creation + usage     |
| 12-full-autonomy-test   | A4       | Dynamic | Comprehensive A4 validation       |

---

## 9. Related Work

**AutoGen** (Wu et al., 2023) provides a conversational multi-agent framework with flexible agent interaction patterns but uses Python-native configuration, limiting portability.

**CrewAI** (Moura, 2024) introduces role-based agent definitions in YAML but lacks formal orchestration primitives, validation rules, and observability specifications.

**LangGraph** (LangChain, 2024) offers state-machine-based agent orchestration with strong state management but is tightly coupled to the LangChain ecosystem.

**MCP** (Anthropic, 2024) standardizes LLM tool access through a client-server protocol. AWP integrates MCP as the tool layer (Layer 2) while adding the remaining six layers.

**A2A** (Google, 2025) defines agent-to-agent communication semantics. AWP's communication layer (Layer 3) is compatible with A2A semantics while embedding them in a broader workflow context.

AWP is distinguished by its completeness (all seven concerns in one format), its formal autonomy taxonomy, its pre-execution validation framework, and its strict runtime-agnosticism.

---

## 10. Limitations and Future Work

### Current Limitations:

- The reference implementation is Python-only. TypeScript and Go implementations are planned but not yet available.
- Performance benchmarking across runtimes has not been conducted. The conformance suite validates correctness, not performance.
- The delegation loop engine's LLM-based validation step introduces non-determinism. Deterministic validation rules mitigate this, but cannot fully replace semantic quality assessment.

- The current packaging format ( `.awp.zip` ) does not include cryptographic signing or provenance attestation.

### Future Work:

- **Multi-runtime benchmarking:** Using identical AWP workflows to compare execution efficiency across Python, TypeScript, and cloud-native runtimes.
- **Workflow registry:** A public registry for discovering, publishing, and versioning `.awp.zip` packages.
- **Visual authoring tools:** Graphical editors for designing AWP workflows with real-time validation feedback.
- **Cryptographic packaging:** Signed `.awp.zip` archives with provenance chains for supply-chain security.
- **Formal verification:** Model checking of AWP workflow properties (deadlock freedom, budget exhaustion bounds, termination guarantees).

---

## 11. Conclusion

The Agent Workflow Protocol addresses the fragmentation problem in multi-agent AI systems by providing a single, declarative, runtime-agnostic format for describing complete workflows. Its 7-layer architecture covers the full stack of concerns from agent identity to observability. The five-level autonomy taxonomy (A0–A4) enables workflows to span from simple prescribed pipelines to fully self-organizing agent teams with recursive delegation and hard budget enforcement.

The 24-rule validation framework guarantees structural correctness before execution, eliminating a class of runtime errors. The reference implementation demonstrates that the protocol is implementable, and the conformance test suite ensures that multiple runtimes can achieve behavioral equivalence.

AWP does not replace existing standards — it composes them. MCP provides the tool layer, A2A informs the communication layer, and framework-specific innovations inspire the orchestration engines. By separating workflow description from runtime execution, AWP enables the portability, benchmarking, and knowledge sharing that the multi-agent ecosystem currently lacks.

The specification, reference implementation, example workflows, and conformance suite are available as open source under the MIT license.

---

## References

1. Anthropic. (2024). Model Context Protocol Specification. <https://modelcontextprotocol.io>

2. Google. (2025). Agent-to-Agent Protocol. <https://github.com/google/A2A>
3. LangChain. (2024). LangGraph: Multi-Actor Applications with LLMs. <https://github.com/langchain-ai/langgraph>
4. Moura, J. (2024). CrewAI: Framework for Orchestrating Role-Playing AI Agents. <https://github.com/joaomdmoura/crewAI>
5. OpenAPI Initiative. (2024). OpenAPI Specification v3.1.0. <https://spec.openapis.org/oas/v3.1.0>
6. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

---

*Corresponding repository: [github.com/veegee82/agent-workflow-protocol](https://github.com/veegee82/agent-workflow-protocol)*

*License: MIT*